

Conditions et Boucles

Ce document détaille le passage de l'exécution interactive en console à la conception de scripts d'administration autonomes. Il aborde la structure d'un fichier .ps1, l'implémentation de la logique décisionnelle (conditions) et le traitement par lots (boucles).

1. Fichier Script (.ps1)

Le pipeline (|) est optimisé pour le traitement de flux d'objets (filtrage via Where-Object, tri via Sort-Object). Cependant, l'automatisation de tâches d'infrastructure complexes nécessite une logique algorithmique séquentielle (tests multiples, gestion d'erreurs, boucles imbriquées) hébergée dans un script.

1.1 Le fichier .ps1

Un script PowerShell est un fichier texte standard portant l'extension .ps1.

Son exécution locale requiert de spécifier explicitement son chemin (relatif ou absolu) par mesure de sécurité :

```
.\MonScriptAdmin.ps1
```

1.2 La politique d'exécution (Execution Policy)

Par défaut, Windows restreint l'exécution de scripts non signés. Pour autoriser l'exécution de scripts locaux en environnement de développement ou d'administration courante, la politique doit être ajustée :

```
Set-ExecutionPolicy RemoteSigned -Force
```

2. Prise de Décision Logique : Les Conditions

Les structures conditionnelles permettent d'orienter le flux d'exécution du script en fonction de l'évaluation d'une expression logique.

2.1 L'instruction if / elseif / else

La structure conditionnelle standard. Le bloc de code entre accolades {} s'exécute uniquement si l'expression entre parenthèses () est évaluée à \$true.

```
$EspaceLibreGo = 12
```

```

if ($EspaceLibreGo -lt 10) {
    Write-Host "CRITIQUE : Espace disque insuffisant." -ForegroundColor Red
}
elseif ($EspaceLibreGo -lt 20) {
    Write-Host "AVERTISSEMENT : Espace disque faible." -ForegroundColor Yellow
}
else {
    Write-Host "OPTIMAL : Espace disque conforme." -ForegroundColor Green
}

```

2.2 L'instruction switch

L'instruction switch remplace avantageusement une succession de clauses elseif lors de l'évaluation d'une variable unique face à de multiples états possibles. Elle améliore la lisibilité et les performances de routage.

```
$StatutService = "Stopped"
```

```

switch ($StatutService) {
    "Running" { Write-Host "Le service est opérationnel." }
    "Stopped" { Write-Host "Le service est arrêté." ; Start-Service -Name "Spooler" }
    "Paused" { Write-Host "Le service est en pause." }
    default { Write-Host "Statut du service inconnu." }
}

```

3. Traitement par Lots : Les Instructions d'Itération (Boucles)

PowerShell propose plusieurs instructions d'itération pour parcourir les collections de données. Une distinction technique stricte doit être faite entre les mots-clés du langage et les cmdlets du pipeline.

3.1 Boucle foreach (Langage) vs ForEach-Object (Pipeline)

- **ForEach-Object (ou %)** : Cmdlet utilisée dans le pipeline. Traite les objets au fil de l'eau (streaming). Optimise l'utilisation de la RAM mais présente une exécution globale plus lente.
- **Instruction foreach** : Mot-clé du langage utilisé dans les scripts. La collection doit être chargée intégralement en mémoire vive. Offre des performances d'exécution nettement

supérieures.

Syntaxe de l'instruction foreach :

```
$Serveurs = @("SRV-AD01", "SRV-FILE01", "SRV-SQL01")

foreach ($Machine in $Serveurs) {
    Write-Host "Initialisation de l'audit pour l'hôte : $Machine"
    # Instructions de traitement de l'instance $Machine
}
```

3.2 La boucle for (Itération par index)

Employée lorsqu'un compteur numérique strict est requis ou que le nombre exact d'itérations est connu à l'avance.

```
# Syntaxe : for (Initialisation ; Condition d'arrêt ; Incrémentation)
for ($i = 1; $i -le 5; $i++) {
    Write-Host "Tentative de connexion réseau (Essai $i/5)..."
    Start-Sleep -Seconds 1
}
```

3.3 La boucle while (Attente d'état)

Exécute un bloc d'instructions tant qu'une condition évaluée reste vraie. Fréquemment utilisée pour les mécanismes de temporisation ou d'attente de disponibilité d'une ressource (monitoring).

```
Write-Host "En attente de la disponibilité du service..."
```

```
while ((Get-Service -Name wuauserv).Status -ne 'Running') {
    Write-Host "." -NoNewline
    Start-Sleep -Seconds 2
}
```

```
Write-Host "`nService disponible. Reprise du script." -ForegroundColor Green
```